

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

GUÍA DE PRÁCTICA EXPERIMENTAL

1 DATOS GENERALES

CARRERA: SOFTWARE (REDISEÑO)

ASIGNATURA: APLICACIONES DISTRIBUIDAS [20701]

NIVEL / PARALELO: 7MO NIVEL – A

N.º DE ESTUDIANTES:

PROFESOR: GUERRERO ULLOA GLEISTON CICERON

PERIODO ACADÉMICO: 2026-2027 PPA

FECHA: 29-04-2026

TIPO DE GUÍA: PRÁCTICA EXPERIMENTAL

TIPO DE UBICACIÓN: INTERNA

TIPO AULA/LABORATORIO: LABORATORIOS

AULA/LABORATORIO: TIC-001

UBICACIÓN: CAMPUS CENTRAL

TIPO DE APRENDIZAJE: Aplicación de contenidos técnicos

N. HORAS: 9

2 DATOS ACADÉMICOS

Resultado de aprendizaje

Aplica arquitecturas y patrones de diseño distribuido —microservicios, API REST, mensajería asincrónica, arquitectura en capas— para construir sistemas escalables, disponibles y seguros.

Tema de la práctica

Diseño, implementación y despliegue de una arquitectura de microservicios REST con autenticación JWT, contenerización con Docker Compose y gestión de peticiones con API Gateway (Nginx), integrada con el Proyecto Fin de Curso.

3 OBJETIVO

Objetivo General:

Implementar una arquitectura de microservicios funcional con tres microservicios independientes, API Gateway y autenticación JWT, contenerizados con Docker Compose, constituyendo la Entrega 2 del PFC.

Objetivos Específicos:

1. Diseñar la arquitectura de microservicios del PFC con diagramas C4 nivel 2 y nivel 3.
2. Implementar tres microservicios REST independientes (auth-service, resource-service, notification-service).
3. Configurar un API Gateway con Nginx que enruta peticiones a cada microservicio.
4. Implementar autenticación JWT y contenerizar el sistema con Dockerfile multi-stage y docker-compose.yml.

4 FUNDAMENTO TEÓRICO

Desarrollar en LaTeX el fundamento teórico COMPLETO de la Unidad 3. Cada subtema: mínimo 300 palabras, al menos 2 referencias IEEE verificadas con DOI, diagrama propio, y código de ejemplo en Istlisting.

2.1 Evolución Arquitectónica: Monolito → SOA → Microservicios

Desarrollar: evolución histórica y técnica, comparación cuantitativa entre arquitecturas (tiempos de despliegue, escalabilidad, aislamiento de fallos), principios SOA, y los 9 principios de microservicios. Incluir tabla comparativa en booktabs.

Referencias bibliográficas mínimas [IEEE]:

- [1] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021. ISBN: 978-1-4920-3402-5.
- [2] G. Blinowski, A. Ojdowska, and A. Przybylek, 'Monolithic vs. microservice architecture: A performance and scalability evaluation,' IEEE Access, vol. 10, pp. 20357-20374, 2022. doi: 10.1109/ACCESS.2022.3152803.

2.2 Patrones de Arquitectura Distribuida: EDA, CQRS y API Gateway

Desarrollar: arquitectura orientada a eventos (EDA) con Kafka y RabbitMQ, patrón CQRS, patrón Saga, y patrón API Gateway. Para cada patrón: definición, cuándo usarlo, ventajas/desventajas y ejemplo.

Referencias bibliográficas mínimas [IEEE]:

[3] U. Joshi, Patterns of Distributed Systems. Boston, MA, USA: Addison-Wesley Professional, 2023. ISBN: 978-0-13-822198-0.

[4] I. Karabey Aksakalli et al., 'Deployment and communication patterns in microservice architectures,' J. Syst. Softw., vol. 180, p. 111014, Oct. 2021. doi: 10.1016/j.jss.2021.111014.

2.3 APIs RESTful: Diseño, Documentación y Seguridad con JWT

Desarrollar: seis principios REST de Fielding, diseño de recursos y URIs, verbos HTTP (seguridad e idempotencia), códigos de estado, documentación OpenAPI 3.0, autenticación JWT (estructura del token, flujo completo), comparación REST vs. gRPC vs. GraphQL.

Referencias bibliográficas mínimas [IEEE]:

[5] R. T. Fielding, 'Architectural styles and the design of network-based software architectures,' Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

[6] M. Jones, J. Bradley, and N. Sakimura, 'JSON Web Token (JWT),' Internet Engineering Task Force, RFC 7519, May 2015. doi: 10.17487/RFC7519.

2.4 Contenerización con Docker y Orquestación con Kubernetes

Desarrollar: diferencias VM vs. contenedores, arquitectura Docker (daemon, imagen, contenedor, registro), Dockerfile y sus instrucciones, Docker Compose multi-contenedor, introducción a Kubernetes (pods, deployments, services, ingress), patrones blue-green y canary.

Referencias bibliográficas mínimas [IEEE]:

[7] B. Burns, Designing Distributed Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2025. ISBN: 978-1-09-815635-0.

[8] C. Richardson, Microservices Patterns: With Examples in Java. Shelter Island, NY, USA: Manning Publications, 2018. ISBN: 978-1-617-29454-1.

5 MATERIALES, EQUIPOS, INSUMOS Y REACTIVOS

Materiales

- TeX Live 2023+ o Overleaf
- Git 2.x + GitHub
- Postman 10.x + Swagger Editor (editor.swagger.io)
- Nginx 1.24 (imagen Docker: nginx:alpine)
- JDK 21 + Spring Boot 3.x — O — Node.js 20 LTS + Express
- Python 3.11+ con FastAPI, uvicorn, python-jose, sqlalchemy.
- Docker Desktop 4.x o Docker Engine + Docker Compose Plugin

Equipos

- Computadora por integrante: iX/Ryzen X, 16 GB RAM recomendado.

Insumos y reactivos

- Conexión a Internet para imágenes Docker.

6 PROCEDIMIENTOS

Paso 1 Diseño de la Arquitectura

- Definir los tres microservicios: auth-service (autenticación/JWT), resource-service (recurso principal del PFC), notification-service (notificaciones/reportes).
- Elaborar diagrama C4 nivel 2 en draw.io o Structurizr. Incluir: microservicios, bases de datos propias, API Gateway y cliente.
- Definir contrato OpenAPI 3.0 de cada microservicio en Swagger Editor. Exportar como YAML en /docs/api/.

Paso 2 Implementación de los Microservicios

- auth-service: POST /auth/login (valida credenciales → retorna JWT con exp. 1h), GET /auth/validate (verifica token).
- resource-service: CRUD completo del recurso del PFC. Cada endpoint verifica JWT en header Authorization: Bearer <token>.
- notification-service: POST /notifications (registra evento cuando se crea/modifica un recurso).
- Cada microservicio tiene su propia base de datos (SQLite o PostgreSQL en Docker).
- Todos los endpoints responden JSON: {status, data, message, timestamp}.

⚠ **Nota:** Tecnología libre: FastAPI (Python), Spring Boot (Java) o Express (Node.js).

Paso 3 Configuración del API Gateway (Nginx)

- Nginx escucha en puerto 8080. Rutas: /api/auth/* → auth-service:8001, /api/resources/* → resource-service:8002, /api/notifications/* → notification-service:8003.
- Configurar rate limiting: máximo 100 peticiones/minuto por IP.
- Logging de todas las peticiones: timestamp, método, ruta, IP, código de respuesta.

Paso 4 Contenerización con Docker Compose

- Dockerfile multi-stage para cada microservicio (etapa builder + runtime).
- docker-compose.yml: tres microservicios + bases de datos + Nginx + red interna (bridge).
- Verificar: 'docker-compose up --build' levanta el sistema sin errores.
- Ejecutar pruebas con Postman: mínimo 3 flujos end-to-end documentados con capturas.

⚠ **Nota:** Incluir .env.example con todas las variables de entorno (sin valores reales en el repositorio).

Paso 5 Documento LaTeX (PFC Entrega 2)

- Incluir: fundamento teórico U3 completo, arquitectura C4 como figura, contratos OpenAPI en Istlisting YAML, Dockerfiles comentados en Istlisting, tabla booktabs de resultados de pruebas (endpoint|método|código esperado|obtenido|latencia ms).
- Mínimo 12 páginas de contenido. Compilar sin errores.

7 BIBLIOGRAFÍA

- R. T. Fielding, 'Architectural styles and the design of network-based software architectures,' Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.
- B. Burns, Designing Distributed Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2025.
- I. Karabey Aksakalli et al., 'Deployment and communication patterns in microservice architectures,' J. Syst. Softw., vol. 180, p. 111014, Oct. 2021. doi: 10.1016/j.jss.2021.111014.
- G. Blinowski, A. Ojdowska, and A. Przybyłek, 'Monolithic vs. microservice architecture,' IEEE Access, vol. 10, pp. 20357-20374, 2022. doi: 10.1109/ACCESS.2022.3152803.
- U. Joshi, Patterns of Distributed Systems. Boston, MA, USA: Addison-Wesley Professional, 2023. ISBN: 978-0-13-822198-0.
- S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021. ISBN: 978-1-4920-3402-5.

8 ANEXOS

- Flujos de prueba obligatorios con Postman
- Flujo 1: POST /api/auth/register → POST /api/auth/login → verificar JWT retornado.
- Flujo 2: GET /api/resources sin token (→ 401) → con token (→ 200) → POST → PUT → DELETE.
- Flujo 3: Crear recurso → verificar notificación en notification-service.
- Preguntas de análisis obligatorias
- ¿Por qué cada microservicio debe tener su propia base de datos? ¿Qué problema resuelve el patrón Database per Service?
- ¿Qué ocurre si resource-service cae? Propone la implementación del patrón Circuit Breaker para este caso.
- Compara el rendimiento con y sin API Gateway. ¿Introduce el API Gateway un cuello de botella? ¿Cómo lo minimizarías?

- ¿Cómo escalarías resource-service a 3 réplicas con Docker Compose? ¿Qué cambiarías en Nginx?

9 RESULTADOS DE ESTUDIANTES

10 RESPONSABLES

GUERRERO ULLOA GLEISTON CICERON
 CI: 0913531752
DOCENTE RESPONSABLE

PONCE ORDOÑEZ JESSICA ALEXANDRA
 CI: 1205316456
COORDINADOR DE CARRERA

